

0.75. Arrivé en (0, 0), le A devient B

Le A part de (10, 0) et suit la croix ; sa position s'affiche en direct. Quand il arrive en (0, 0), il se transforme en B : `if (x == 0 && y == 0)`.

1. Ce qu'il faut comprendre

Le A part de (10, 0), en haut de l'écran, et se déplace avec la croix, comme au 0.71. Sa position s'affiche **en direct** en bas de l'écran, comme au 0.68 : `x` à gauche, `y` à côté.

Ce qui est nouveau ici : une condition sur la position. `if (x == 0 && y == 0)` veut dire : « si `x` vaut 0 **ET** si `y` vaut 0 ». `&&` se lit « et » : il faut que **les deux** soient vrais en même temps. En (0, 5), `x == 0` est vrai mais `y == 0` est faux : rien ne se passe. En (0, 0), les deux sont vrais : la condition est vraie.

La lettre dans une variable : `lettre` vaut 0 au départ, et la croix pose `ALPHABET[lettre]`, donc `ALPHABET[0]`, le A. Quand la condition est vraie, `lettre = 1;` : à l'image suivante, `deplace_croix` pose `ALPHABET[1]`, le B.

Le B reste un B : rien ne remet `lettre` à 0. Même en quittant la case (0, 0), la lettre reste transformée.

Deux façons d'écrire le `if` : sur une ligne, `if (x == 0 && y == 0) lettre = 1;`, ou **avec des accolades** : `if (x == 0 && y == 0) { lettre = 1; }`. Les deux font exactement la même chose. Sans accolades, le `if` ne commande **qu'une seule** instruction, celle qui le suit. Avec des accolades, il commande **tout ce qui est entre elles** : on peut y mettre plusieurs lignes (changer la lettre ET jouer un son, par exemple). Le programme l'écrit avec des accolades, et montre l'autre forme en commentaire juste au-dessus.

Pour y arriver : la flèche **gauche**, dix pas. Les nombres en bas montrent `x` qui descend : 010, 009... jusqu'à 000. Et la lettre change.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1822 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;    // la COLONNE : le A part de la colonne 10...
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;     // ...et de la ligne 0, en haut de l'écran
```

3. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0; // QUELLE lettre : 0 = A, 1 = B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

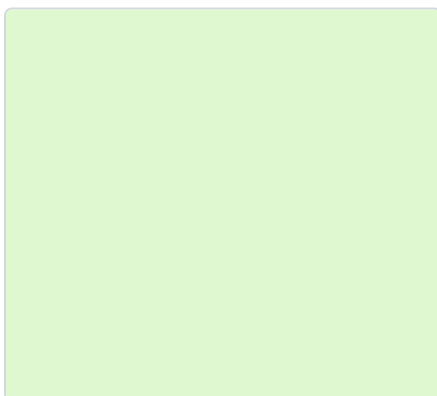
        // La croix déplace la lettre ; ALPHABET[lettre] : le A, puis le B.
        deplace_croix(x, y, ALPHABET[lettre], 250);

        // Le contrôleur de position : x et y, en direct, en bas de l'écran.
        nombre(0, 17, x);
        nombre(4, 17, y);

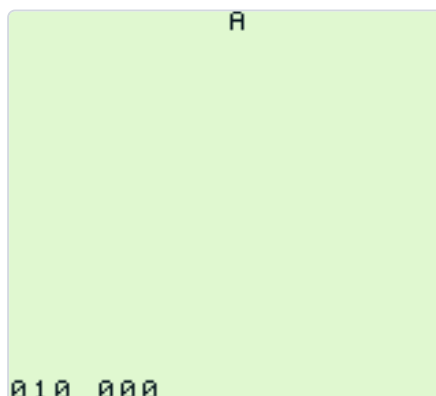
        // La condition, morceau par morceau :
        //
        //   if (x == 0 && y == 0) lettre = 1;
        //   |         | |         |
        //   |         | |         +-- alors : la lettre devient ALPHABET[1], le B
        //   |         | +----- y vaut-il 0 ? (tout en haut)
        //   |         +----- && : ET – les DEUX doivent être vrais
        //   +----- x vaut-il 0 ? (tout à gauche)
        //
        //   en (10, 0) : x == 0 faux          → rien
        //   en (0, 5)  : x == 0 vrai, y == 0 faux → rien
        //   en (0, 0)  : les deux vrais        → le A devient B
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A en haut de l'écran ; avec la flèche gauche, il va jusqu'au coin (0, 0), et là il devient un B. En bas, sa position en direct.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
 - ✓ en route, c'est encore un A ($x \neq 0$)
 - ✓ en (0, 0), il est devenu B
 - ✓ il reste un B en repartant (en (0, 3))
 - ✓ la position s'affiche
-

Ensuite : **0.75.1. Arrivé en (0, 0), le A devient B — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++